

Custom RISC-V Instruction Extensions for Edge Video Anomaly Detection

Design Presentation

Eapen Luke
U2303087

Devamanas S
U2303082

Junit Sarah Varghese
U2303129

Kevin Jose
U2303131

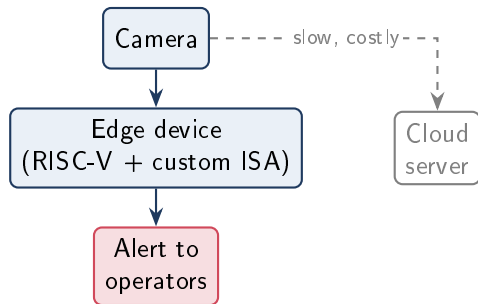
Guide: Dr. Jomina John

Department of Computer Science and Engineering

- ➊ Introduction
- ➋ Problem Definition
- ➌ Objectives
- ➍ Functional & Non-Functional Requirements
- ➎ Overall System Architecture
- ➏ Modules and Their Details
- ➐ Design Diagrams
- ➑ Workloads & Test Data
- ➒ Risks & Challenges
- ➓ Expected Output
- ➔ Conclusion
- ➕ References

Introduction: Why Edge AI for Surveillance?

- Metro / CCTV networks need **continuous, real-time** anomaly detection: violence, falls, crowd panic, fire, abandoned luggage.
- Sending every frame to a server costs **latency, bandwidth and privacy**.
- **Edge inference**: run the neural network on a small device next to the camera.
- Edge CPUs are slow at the core CNN operation, the **multiply-accumulate (MAC)**, which runs millions of times per frame.



Introduction: RISC-V and the Shakti C-Class Core

RISC-V

- Open, royalty-free instruction set architecture (ISA)
- Opcode space reserved for **custom instructions** (custom-0..3)
- Lets us add hardware operations tailored to a workload

Shakti C-Class (IIT Madras)

- 64-bit **RV64IMAFDC**, in-order, 6-stage pipeline
- Hardware FPU (Berkeley HardFloat), caches, MMU, AXI4 bus
- Written in **Bluespec SystemVerilog**, a **soft core** (can be simulated or put on an FPGA)
- Chosen over PicoRV32: a real pipeline, Linux-capable, and extensible

Simulation flow: BSV $\rightarrow$ bsc $\rightarrow$ Verilog $\rightarrow$ Verilator $\rightarrow$ cycle-accurate model bin/out

Problem Definition

Problem Statement

Design **workload-driven custom RISC-V instructions**, integrate them into the Shakti C-Class core to speed up **edge video anomaly detection**, measured against the unmodified baseline core.

Objectives

- ➊ **Profile** the target workloads (MobileNetV2 + GRU, then TinyMetro-Net) on PC *and* on the Shakti core itself.
- ➋ **Identify** the dominant kernels and the real hardware bottleneck.
- ➌ **Design** custom instructions in RISC-V custom opcode space.
- ➍ **Implement** them in the Shakti decoder and floating-point unit (RTL).
- ➎ **Verify** bit-exact results and backward compatibility (official riscv-tests).
- ➏ **Measure** speedup using hardware counters (`mcycle`, `minstret`).
- ➐ **Extend** the instruction set to INT8 models (TinyMetro-Net from Team 1).
- ➑ **Run** the complete network on the customised core and **compare** it with the baseline core and a PC/server.

Functional & Non-Functional Requirements

Functional

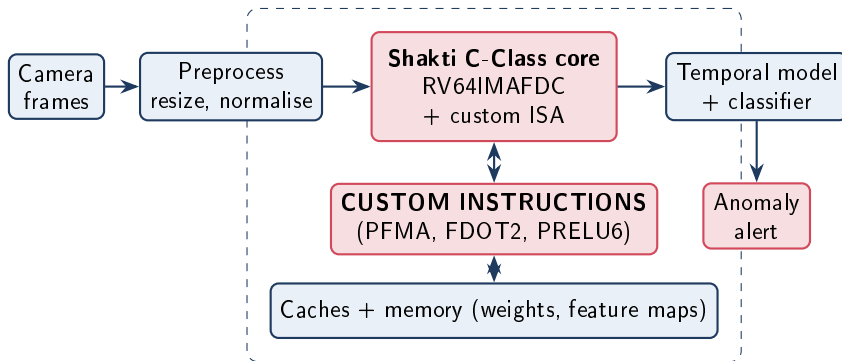
- Decode and execute new custom-opcode instructions
- Results **bit-identical** to standard RISC-V code
- Standard RV64IMAFDC behaviour unchanged
- Per-kernel cycle and instruction measurement
- Run the CNN kernels (conv, depthwise, pointwise, ReLU) on the core
- Support INT8 models (TinyMetro-Net) (*planned*)

Non-Functional

- **Performance:** $\geq 1.5 \times$ kernel speedup
- **Correctness:** riscv-tests (rv64uf-fmadd) pass
- **Low overhead:** small hardware additions
- **Reproducibility:** open-source flow (bsc, Verilator, GCC 14)
- **Edge constraints:** limited memory (16 KB D-cache), real-time frame rate
- **Maintainability:** documented, logged design decisions

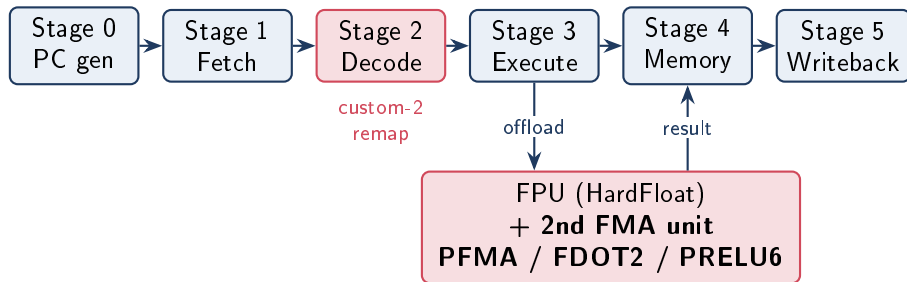
Overall System Architecture

Shakti SoC (Verilator simulation; FPGA-ready soft core)



The CNN kernels (convolutions, activations) run on the core; the custom instructions accelerate them inside the floating-point unit.

Architecture: Inside the Shakti C-Class Pipeline



- **Modified**: `decoder.bsv`, `decoder.defines`, `fpu_hardfloat.bsv`
- **Unchanged**: `fetch`, `scoreboard`, `forwarding`, `memory`, `writeback`. The new instructions reuse the existing `FLOAT` path.

Module 1: Workload Profiling (PC → Shakti)

PyTorch on a PC (MobileNetV2 + GRU)

Operation	MACs / frame
Pointwise 1×1	267.9 M
Depthwise 3×3	20.7 M
First conv 3×3	10.8 M
GRU (8 steps)	9.4 M

Measured on baseline Shakti (1 inference)

Operation	cyc/MAC	Share
Pointwise	11.7–13.9	83.8%
Depthwise	16.7	8.7%
First conv	17.7	4.8%
ReLU6	14.3/elem	2.2%
GRU	12.0	0.4%
Total	31.9 B cyc	

Finding

Pointwise convolution dominates on the real target. The FP32 data type rules out an INT8-only design, and Shakti already has `fmadd.s`, so a scalar custom MAC would add nothing.

Module 2: Custom Instruction Design

Instruction	Type	What it does	Status
FDOT2.S	FP32	$fd = a_1 b_1 + (a_0 b_0 + c)$: 2-element dot product	✓ done
PFMA.S	FP32	$fd = [x_1 w + c_1 \mid x_0 w + c_0]$: 2 FMAs in parallel	✓ done
PRELU6.S	FP32	clamp both packed values to $[0, 6]$ in 1 cycle	✓ done
PDOT8	INT8	8 INT8 multiply-accumulates in one instruction (custom-1)	✓ tested
PFADD.S	FP32	packed add of 2 values (residual / skip connections)	proposed
PFMAX.S	FP32	packed max of 2 values (max pooling)	proposed
PSIGM / PTANH	FP32	fast sigmoid / tanh approximation (GRU gates)	proposed
FLD.PI	memory	load + pointer post-increment (removes address updates)	proposed
PREQ	INT8	requantise + ReLU: $\text{clip}((acc \cdot s) \gg 15, 0, 127)$	proposed
PMAX8	INT8	packed INT8 max (pooling)	proposed

- **Encoding:** FP32 instructions use **custom-2** (0x5B, R4 layout, fmt bits [26:25]); INT8 instructions use **custom-1** (0x2B, funct3) in the integer ALU.
- **Packing:** two FP32 values (or eight INT8 values) in one 64-bit register.
- **Selection rule:** add an instruction only where the **Shakti profile** shows the time goes (pointwise 84%, depthwise 9%, first conv 5%, ReLU6 2%, GRU 0.4%).

Module 3: Hardware Integration (RTL)

- ❶ **Decoder remap**: rewrite custom-2 into an internal FMADD form, so the existing decode, scoreboard and writeback are reused unchanged.
- ❷ **Own FPU opcode (fn)**: the R4 f7 field holds register data, not instruction bits.
- ❸ **Raw operands**: bypass NaN-boxing for packed values.
- ❹ **Second FMA unit** spfma2 for PFMA lane 1.
- ❺ **PRELU6**: integer compares on IEEE bits, 1 cycle.

Verification

- Directed test: FDOT2 of $[2, 1.5] \cdot [4, 3] + 0.5 = 13.0$ ✓
- Every benchmark checks **bit-exact** equality with the baseline C code
- Official rv64uf/fmadd.S passes after each change
- Pass/fail read from tohost; traces in rtl.dump

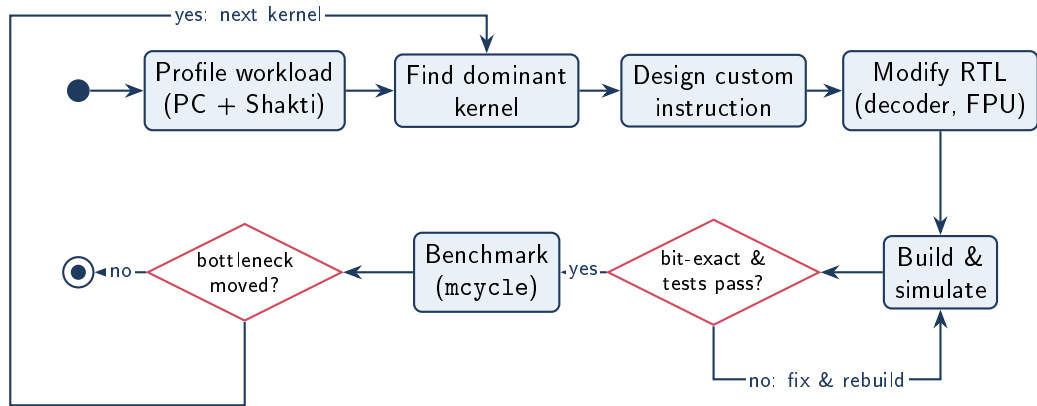
Benchmark flow

C kernel $\rightarrow$ RISC-V GCC $\rightarrow$ elf2hex $\rightarrow$ simulated Shakti $\rightarrow$ mcycle/minstret $\rightarrow$ report

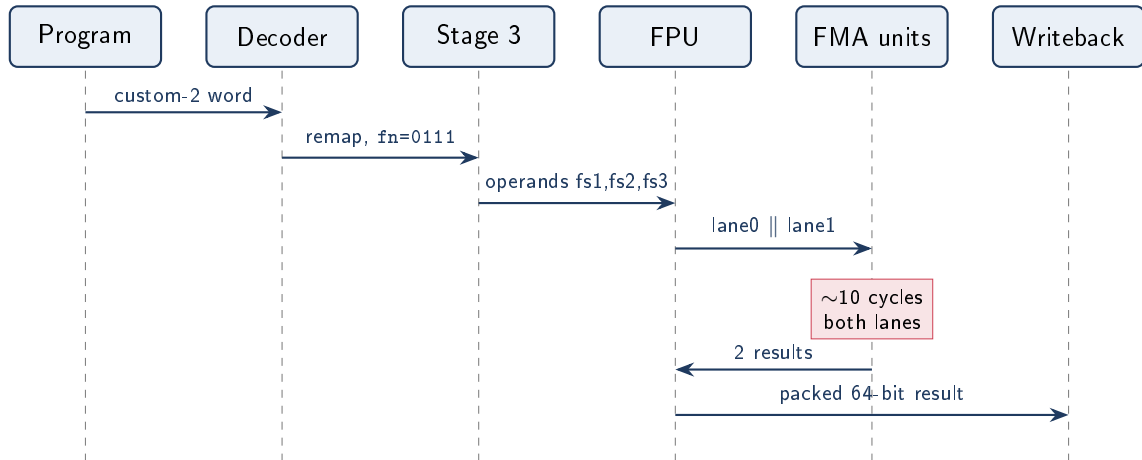
Measured Results (all bit-exact)

Kernel	Baseline	Custom	Speedup
Pointwise, small K (16–144 ch)	11.1–11.8 cyc/MAC	6.6–7.0	1.69–1.70 ×
Pointwise, large K (320–960 ch)	13.8–13.9 cyc/MAC	9.2–9.3	1.49–1.50 ×
Depthwise 3×3 (with pair copy)	16.99 cyc/MAC	10.86	1.56 ×
First conv 3×3, stride 2	18.28 cyc/MAC	9.22	1.98 ×
ReLU6 (PRELU6)	14.00 cyc/elem	3.52	3.98 ×
Whole MobileNetV2 + GRU inference (estimate)		Cycles	Speedup
Baseline Shakti		31.9 B	1.00×
+ PFMA (pointwise, depthwise, first conv)		19.8 B	1.61×
+ PRELU6		19.3 B	1.66 ×

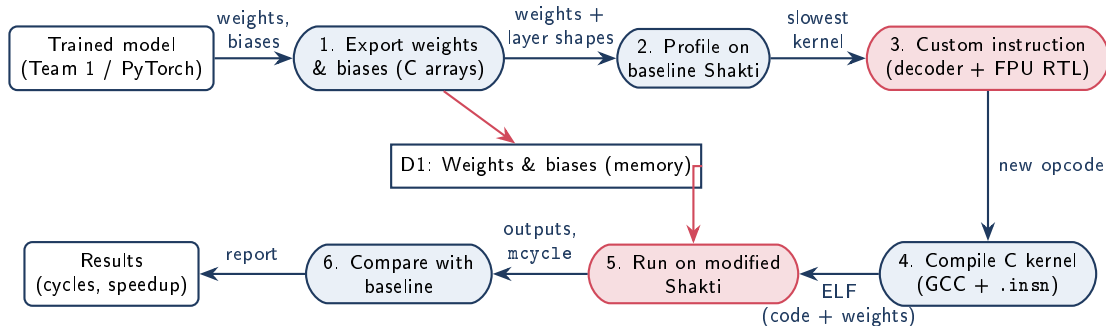
Activity Diagram: Workload-Driven Design Method



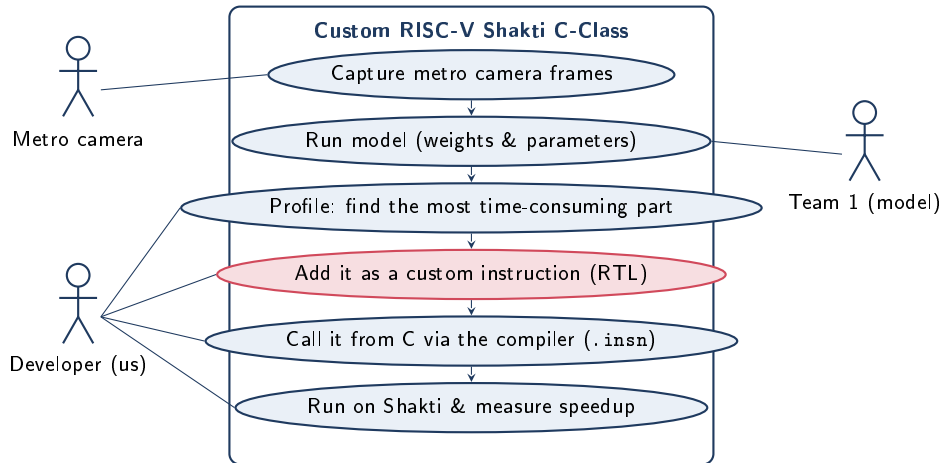
Sequence Diagram: Executing One PFMA.S Instruction



Data Flow Diagram (Level 1): Custom Instruction Flow



Use Case Diagram



Workloads & Test Data

Workload	Data type	Data used
MobileNetV2 + GRU	FP32	ImageNet-pretrained MobileNetV2; 8-frame sequences of 224×224 ; random inputs for timing
TinyMetro-Net (Team 1)	INT8	8-class metro dataset: 224 test frames, 527 validation frames, 54 held-out clips

- Cycle timing does not depend on the data values, so profiling uses real layer shapes with synthetic data.
- Correctness is checked bit for bit against the baseline implementation.

Risks & Challenges

- **Non-pipelined FPU:** one operation at a time limits the total speedup to $\sim 1.66\times$ (Amdahl's law)
- **Simulation speed:** a full network takes days in Verilator, so we measure sampled layers and extrapolate
- **Memory:** the 16 KB D-cache slows the large layers
- **FPGA prototype:** resource usage of the modified core on an FPGA is not yet measured
- **Software port:** the full network must be written in bare-metal C (no OS, no Python)
- **Data type:** target models are INT8, while our first instructions are FP32
- **Tooling:** stale builds, long rebuilds (15–20 min)

Expected Output

Deliverable	Status	Result
PyTorch + Shakti profiling	✓	pointwise = 84% of the time
FDOT2.S	✓	$1.87\times$ fewer instructions, $1.07\text{--}1.13\times$
PFMA.S (2nd FMA unit)	✓	$1.5\text{--}2.0\times$ on conv kernels
PRELU6.S	✓	$3.98\times$ on ReLU6
Regression (riscv-tests)	✓	all pass, bit-exact
Whole-model estimate	✓	31.9 B $\rightarrow$ 19.3 B cycles ($1.66\times$)

- A **workload-driven** method (profile $\rightarrow$ bottleneck $\rightarrow$ instruction $\rightarrow$ verify $\rightarrow$ measure) produced real, measured gains on an open RISC-V core.
- Four custom instructions (FDOT2, PFMA, PRELU6, PDOT8) were designed, built into Shakti C-Class and verified **bit-exact**, with standard ISA behaviour unchanged.
- Estimated **1.66 $\times$** speedup for MobileNetV2 + GRU; the remaining bottleneck (the non-pipelined FPU) is identified and quantified.
- Next: a pipelined FPU, INT8 instructions for Team 1's TinyMetro-Net, and running the complete network on the customised core.

References I

- ❶ A. Waterman, K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International.
- ❷ SHAKTI Processor Program, IIT Madras, *C-Class Core Documentation*, gitlab.com/shaktiproject/cores/c-class.
- ❸ M. Sandler *et al.*, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *CVPR*, 2018.
- ❹ K. Cho *et al.*, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
- ❺ J. Hauser, *Berkeley HardFloat* floating-point hardware library.
- ❻ R. Nikhil, *BSV by Example*, Bluespec Inc.
- ❼ W. Snyder, *Verilator* open-source SystemVerilog simulator.
- ❽ J. Hennessy, D. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.
- ❾ Team 1, *TinyMetro-Net: Architecture & Hardware Specification*, internal report, v1.0.

Thank You